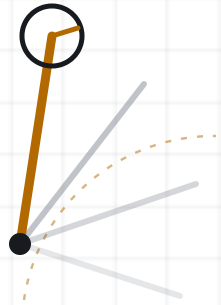


Swinging It Up

Getting the rod from a dead hang to balanced upright — energy control, the catch, and why they're two different jobs.



Convention: θ from straight DOWN · $\theta=0$ hangs · $\theta=\pi$ upright

You've built a controller that **balances** the rod near the top. Now you want to **get it there** from a dead hang. The single most important idea, planted before anything else:

Swing-up and balancing are two different jobs done by two different controllers, joined by a switch.

You half-suspected LQR was a swing-up method. It isn't. Let's fix that, then build the real swing-up controller from the ground up.

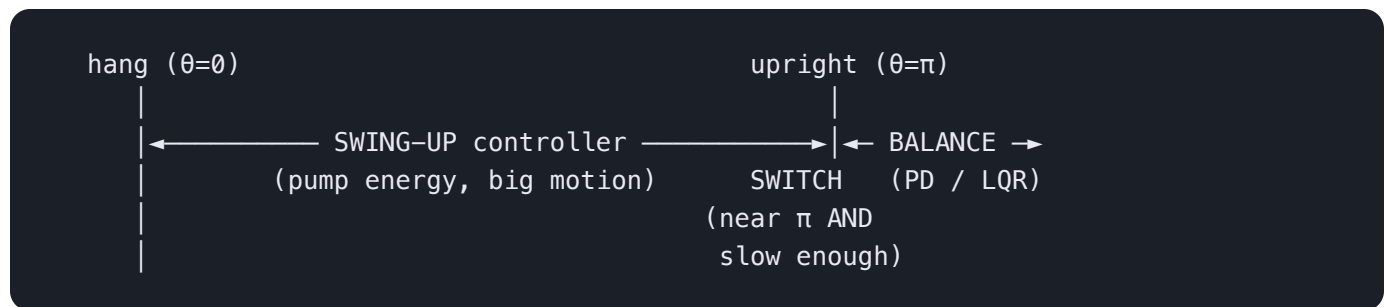
SECTION 01

Two jobs, one switch

Near the top ($\theta \approx \pi$): the balancing world. The rod is almost upright; you hold it against gravity and nudge out small wobbles. The physics here is *almost a straight line*, so simple feedback works beautifully. This is where your gravity-feedforward + PD lives, and where LQR lives. LQR isn't a new capability — it's a smarter cousin of the PD you already wrote.

Everywhere else (θ from 0 toward π): the swing-up world. The rod travels a huge arc, gains speed at the bottom, and must arrive at the top nearly stopped. This motion is wildly *curved* (nonlinear). A balancer is useless here — it's built on a straight-line approximation honest only within a few degrees of the top. Point a balancer at a hanging rod and it computes torque as if the rod were "180° from where I assume you are." Garbage, and it'd instantly slam your wheel.

The architecture is a **relay race**:



The swing-up controller's only real job is to **deliver the rod into the balancer's catch window** — close enough in angle *and* slow enough to be caught. Then a switch hands over.

SECTION 02

The main event: energy-based swing-up

2.1 · What "the pendulum's energy" is

Kinetic energy (motion): $KE = \frac{1}{2}J\dot{\theta}^2$, where θ is how fast the arm rotates and J is your pivot inertia (`arm_moment`). Squared speed → always positive, **zero exactly when the arm is momentarily stopped**. (Rotational twin of $\frac{1}{2}mv^2$.)

Potential energy (height): $PE = K_g \cdot (1 - \cos \theta)$, where $K_g = (m_a \cdot \ell / 2 + m_w \cdot L) \cdot g$ is your gravity stiffness. At the hang ($\theta=0$): $1 - \cos 0 = 0$, no height energy. At the top ($\theta=\pi$): $1 - \cos \pi = 2$, maximum.

$$E = \frac{1}{2}J\dot{\theta}^2 + K_g \cdot (1 - \cos \theta)$$

As the arm swings, KE and PE seesaw. **E is the single number tracking "how big is this swing," no matter where you looked.**

WHY THIS EXACT FORMULA?

With the motor off, E must stay *constant*. Your dynamics: $J \cdot \ddot{\theta} = -K_g \cdot \sin \theta - \tau$. With $\tau=0$:
 $dE/dt = \dot{\theta} \cdot (J \cdot \ddot{\theta} + K_g \cdot \sin \theta) = \dot{\theta} \cdot (-K_g \cdot \sin \theta + K_g \cdot \sin \theta) = 0$. ✓ The gravity terms cancel — this is the right energy for your machine. (A $1 + \cos \theta$ version would *not* stay constant and puts max PE at the bottom — backwards. Don't use it for your code.)

2.2 · Upright is really a target ENERGY, not a target angle

To arrive at the top *just barely* (stopped), you need exactly enough energy to have lifted the mass all the way up with nothing left for speed:

$$E_0 = K_g \cdot (1 - \cos \pi) = 2 \cdot K_g$$

So "get to upright" becomes "get the total energy up to E_0 ." An angle target is a single hard-to-hit point (right place, right instant, zero speed). An energy target is a **budget you fill gradually, swing by swing**. That's what makes the impossible-looking problem tractable.

2.3 · Pumping: how you add energy

Like a kid on a swing — a little push *timed with the motion*, every pass, and the arc grows: push **with** the motion → **adds** energy; push **against** it → **removes** energy (brakes).

But your motor can't push the arm directly — it spins the wheel, and the arm feels the reaction $-\tau$ (your `-1 * self.wheel_torque`). So "pumping" = **spin the wheel the right way at the right moment so its reaction kick pushes the arm the way it's already going**.

2.4 · The one equation that runs the whole show

With the motor on, keep the $-\tau$ term:

$$dE/dt = \dot{\theta} \cdot ((-K_g \cdot \sin \theta - \tau) + K_g \cdot \sin \theta) = -\tau \cdot \dot{\theta}$$

In words: energy inflow = reaction torque $\times$ arm speed. This is just **power = torque $\times$ angular velocity**. Two things matter:

1. **The gravity terms cancelled $\rightarrow$ there is NO $\cos \theta$ factor in your rule.** If you've seen Åström–Furuta's swing-up written with $\text{sign}(\dot{\theta} \cdot \cos \theta)$, that cosine is from *their* actuator — they shove the pivot sideways, so only a horizontal projection couples in. Yours is a direct reaction torque, so the coupling is clean, no cosine. Don't copy theirs.
2. It tells you exactly how to choose torque.

2.5 • The control law and pump/brake logic

You want $dE/dt > 0$ when $E < E_0$ and < 0 when $E > E_0$. Since $dE/dt = -\tau \cdot \dot{\theta}$:

$$\tau = k \cdot (E - E_0) \cdot \text{sign}(\dot{\theta}) \quad (k > 0)$$

Plug back in: $dE/dt = k \cdot (E_0 - E) \cdot |\dot{\theta}|$ (since $\text{sign}(\dot{\theta}) \cdot \dot{\theta} = |\dot{\theta}|$).

- $E < E_0$: $(E_0 - E) > 0 \rightarrow$ energy rises $\rightarrow$ **pump** ✓
- $E > E_0$: flips negative $\rightarrow$ energy falls $\rightarrow$ **brake** ✓

That one line automatically fires the wheel one way while the arm swings left, the other way while it swings right — you never hand-time anything. $\text{sign}(\dot{\theta})$ picks direction; $(E - E_0)$ sets strength.

Practical forms: **bang-bang** $\tau = \tau_{\max} \cdot \text{sign}((E - E_0) \cdot \dot{\theta})$ (fastest, chatters), or **saturated-proportional** $\tau = \text{saturate}(k \cdot (E - E_0) \cdot \text{sign}(\dot{\theta}), \tau_{\max})$ (the practical one). $\text{sign}(\dot{\theta})$ is undefined at $\dot{\theta}=0$ — define it 0 there or add a deadband so the wheel doesn't buzz at turning points.

VERIFY, DON'T TRUST

I derived the sign from your dynamics, but confirm it — balancer OFF, pump ON, plot $E(t)$.
Marches to $E_0 \rightarrow$ sign right. Runs away $\rightarrow$ flip the sign of k .

2.6 · Why amplitude walks up – and self-limits

Every pass adds a little, so the swing goes higher — like the kid. **But not forever:** when E reaches E_0 , the error $(E - E_0) \rightarrow 0$, the pump command $\rightarrow 0$, it *stops adding*. Overshoot a hair and it *brakes* next pass. E_0 is a **ceiling it converges to, not a speed it keeps piling on**. That self-limiting is what makes it robust.

2.7 · Why you MUST switch controllers near the top

Energy control drives **one number** E to **one value** E_0 . But infinitely many states have $E = E_0$: stopped-at-top (what you want), swinging *through* the top at speed, at 170° moving fast... Picture a contour line on a topo map: $E = E_0$ is a **whole loop of states**, and energy control slides you freely along it — it has no reason to prefer stopped-at-the-top.

Physically the top is unstable (pencil on its tip); regulating total energy does nothing to resist a lean (leaning left vs right at the same speed/height is the *same energy*). Worse, near the top $E \approx E_0$, so the pump command **relaxes to nearly zero exactly when you need a strong restoring push**. Energy control delivers the rod to the top *passing through*, but can never clamp it there. Hence the switch.

SECTION 03

The catch: balancing at the top

3.1 · Why "near the top" is a straight-line world

Zoom into any smooth curve far enough and it looks straight. Near the top we replace $\sin \theta$'s curve by its tangent — **linearizing** — honest only within a few degrees. Let $\delta = \theta - \pi$. Using $\sin(\pi + \delta) \approx -\delta$:

$$J \cdot \ddot{\delta} = -K_g \cdot \sin(\pi + \delta) - \tau = +K_g \cdot \delta - \tau \quad \Rightarrow \quad \ddot{\delta} = (K_g/J) \cdot \delta - \tau/J$$

The $+(K_g/J) \cdot \delta$ term is a **runaway** — gravity's positive feedback (tilt a hair, it pulls you further, faster; ~ 82 ms doubling time on the example rig). The $-\tau/J$ is your authority fighting it. This runaway is why balancing needs active feedback and why the recoverable basin is bounded.

3.2 · How your PD relates to LQR

Your balancer: $\tau = K_P \cdot (\theta - \pi)$ [stiffness] + $K_D \cdot \dot{\theta}$ [damping] + arm_torque [gravity feedforward].

LQR does the same job but tunes itself and adds a term you're missing. First, the **state** — the smallest list predicting the future:

$$\mathbf{x} = [\delta, \dot{\delta}, \omega] = [\text{lean}, \text{lean rate}, \text{wheel speed}]$$

(Wheel *angle* isn't in it — physics only cares how fast the wheel spins.) LQR's law:

$$\tau = -(k_1 \cdot \delta + k_2 \cdot \dot{\delta} + k_3 \cdot \omega)$$

$k_1 \approx$ your K_P , $k_2 \approx$ your K_D , and k_3 is **new: push back proportional to wheel speed**. You don't hand-tune the k 's. You write a **cost** trading state error vs torque spent:

$$\text{cost} = \Sigma (q_1 \cdot \delta^2 + q_2 \cdot \dot{\delta}^2 + q_3 \cdot \omega^2 + R \cdot \tau^2)$$

Big q / small R = aggressive; big R = gentle. **Same stiffness-vs-effort trade you make by hand — just explicit and multi-dimensional.** A standard "Riccati solver" (a black box you call) reads the linearized dynamics + your cost and returns the gains that provably minimize it. You tune the cost; the solver computes the gains. (Don't expect $k_1 = K_P = 5$ — your feedforward already carries part of it, and sign conventions differ. Same *role*, not same number.)

3.3 · The one thing LQR buys that PD can't: wheel-speed regulation

Wheel speed ω is a **pure integrator of torque** — every torque you ever command piles onto ω and nothing pulls it back. Your PD **never reads ω** , so it'll balance the *arm* perfectly while the *wheel silently spins up to saturation* — then back-EMF steals your torque and the catch collapses, invisibly from the arm's telemetry. The fix is one weight: $q_3 > 0 \rightarrow$ LQR assigns $k_3 > 0 \rightarrow$ the $-k_3 \cdot \omega$ term **actively brakes the wheel toward zero**. This is the single biggest thing LQR gives you over your PD. (You could bolt just this onto your PD — add a $-k_3 \cdot \omega$ term — which is LQR telling you what PD was missing.)

3.4 · The switch: when to hand off

Hand off only when the rod is inside the balancer's **basin** — three axes, and beginners forget the last two:

$$|\theta - \pi| < \theta_{\text{win}} \quad \text{AND} \quad |\dot{\theta}| < \dot{\theta}_{\text{win}} \quad \text{AND} \quad |\omega| < \omega_{\text{headroom}}$$

- **Angle** ($\sim 15\text{--}20^\circ$): near enough that the straight-line model is honest.
- **Speed**: slow enough to arrest before it blows past — *the one people miss* (this is why energy control aims for exactly E_0 : right energy at the top $\Rightarrow$ near-zero speed there).
- **Wheel-speed headroom** (reaction-wheel-specific): arrive with the wheel near saturation and the balancer has no accelerating torque left — it drops the rod *even from a perfect angle and speed*.
Deliver the rod with the wheel SLOW, not just the rod slow.

Map the basin empirically (start the balancer alone from many states near the top, see which it saves). Add hysteresis (wider exit than entry) to avoid chatter, and a fallback (if the balancer is losing it, switch back to swing-up). Blend torques over a few ms once it works — polish, not required.

SECTION 04

The rest of the landscape

Briefly and honestly — energy-based is the one to build; know what else exists.

- **Bang-bang pumping** — crude cousin of energy control (full torque in the energy-adding direction, flip at turning points). Simplest, fastest, near-optimal for a hard single swing; chatters, overshoots (needs the switch). Fine first thing to see pumping work.
- **Heuristic angle-PID pumping — the trap**. Pointing a PID at $\theta=\pi$ from the hang *is literally what your original cell-5 code did, and it cannot swing up under a real torque limit* — a PID pushes "toward π " but can't *accumulate* energy over swings. It rises a little, gravity wins, stalls. Swing-up is *pumping energy*, not *muscling angle*.

- **Sine / resonance pumping** — hackiest. Drive at the natural frequency $f_n = (1/2\pi) \cdot \sqrt{K_g/J}$ and let resonance build. Fatal flaw: frequency drops as amplitude grows, so a fixed drive detunes exactly when it starts working. Use for an hour to *see* resonance / measure f_n , then abandon.
- **Partial Feedback Linearization (Spong)** — model-based twin of energy control, built for *your exact machine*; uses your equations to cancel nonlinear terms. Small step up; still doesn't truly respect the wheel-speed ceiling.
- **Trajectory optimization / direct collocation** — first method that *honestly* respects the wheel-speed ceiling. Chop swing-up into ~ 100 slices, hand a solver the physics + endpoints + $|\tau| \leq \tau_{\max}$ AND $|\omega| \leq \omega_{\max}$ at every slice; minimize time $\rightarrow$ the **fewest-swing plan your hardware permits** (you'll see the "wind up, brake to recover authority, pump again" dance). Offline/open-loop, so add light tracking. High-payoff next step after energy control.
- **MPC** — trajectory-opt re-solved live each tick. Online robustness, hard limits, swing-up + balance in one — but heaviest machinery, and in a clean sim its edge over trajopt is small. For disturbances / hardware.
- **Reinforcement learning** — learn a policy by trial and error. Most machinery, most opaque, teaches you least about your system; respects limits only softly. A "someday, to learn RL" side quest, not sensible for a rig you have exact equations for.

SECTION 05

The reaction-wheel reality check

A reaction wheel has no ground to push against. The arm feels torque only via Newton's third law: motor speeds the wheel, wheel shoves back. Every bit of arm-torque is paid for by throwing angular momentum into the wheel.

That's your **momentum budget**. Three limits (all of which you're modeling):

1. **Torque clamp** $|\tau| \leq \tau_{\max}$ — the saturated pump respects it automatically. Each swing adds *bounded* energy $\Rightarrow$ generally **several swings**.

2. **Back-EMF — torque droops with speed:** $\tau_{\text{available}} \approx \tau_{\text{stall}} \cdot (1 - \omega/\omega_{\text{max}})$ in the *accelerating* direction; braking stays available. This kills the single brute-force shove: a long one-way push piles up wheel speed *and* watches its own authority collapse right at the top. **Now see why pumping is the right shape:** the torque direction **flips every half-swing**, so the wheel winds up then *unwinds* — its speed **sloshes around zero instead of piling up**. The constraints don't just forbid the shove; they *reward* multi-swing pumping.
3. **Wheel friction** leaks energy and momentum — mostly a nuisance, but a small *free* helper pulling ω toward zero.

ANTI-WINDUP, REFRAMED

Your PD has no I-term, but your **wheel is a physical integrator** ($I_w \cdot \omega =$ running sum of every torque). If the pump keeps asking for more than τ_{max} in *one* direction, the surplus accumulates as one-directional wheel speed $\rightarrow$ saturation $\rightarrow$ lost authority. Anti-windup here = momentum management: when saturated in a direction that winds the wheel further, back off k or skip that half-swing. Healthy signature: brief *alternating* saturations that cancel over a swing, ω wagging around zero.

Watch in your sim: plot the wheel-speed trace during swing-up. Healthy = wags around zero, grows slowly. Sick = ramps monotonically toward ω_{max} (pump one-sided, k too high). Cure is usually lower k (gentler, more swings). And time the catch on a low-wheel-speed pass.

SECTION 06

Your recommended learning path

1. **Get the energy signal right (no control).** In your open-loop sim, compute $E = \frac{1}{2} \cdot J \cdot \dot{\theta}^2 + K_g \cdot (1 - \cos \theta)$ each step and plot it — **confirm it stays flat**, and equals $2 \cdot K_g$ at the top. If it drifts, fix that before anything else.
2. **Build the pump, balancer OFF.** $\tau = \text{satrate}(k \cdot (E - E_0) \cdot \text{sign}(\theta), \tau_{\text{max}})$ from the hang. Watch $E(t)$ climb to E_0 — **your sign self-check** (runs away $\rightarrow$ flip k 's sign). Deadband at $\theta=0$. Tune k low $\rightarrow$ up $\rightarrow$ back off; watch the wheel-speed trace wag, not ramp.

3. **Add the switch, hand off to your existing PD** (no LQR needed yet). Map the basin (PD alone from a grid near the top), set the windows just inside the caught cloud. Run hang $\rightarrow$ pump $\rightarrow$ switch $\rightarrow$ balance; log *which swing* catches and *wheel speed at catch*.
 4. **Diagnose:** failing on angle/speed $\rightarrow$ arrive slower; failing from a good angle/speed $\rightarrow$ **check the wheel-speed axis** (wound-up wheel), lower k .
 5. **Upgrade the balancer to LQR (optional)**. Add wheel-speed regulation (§3.3) — the fix for the drift you'll be seeing.
 6. **Reach further only if you want:** direct-collocation trajectory opt (also gives the min-time benchmark free); MPC / RL for disturbances / hardware / learning those techniques.
-

TEST YOURSELF

Checkpoint questions

- 1 **The cosine question.** Åström–Furuta's law has a $\cos \theta$; yours doesn't. Walk through the dE/dt derivation and explain, in terms of *what cancels*, why no cosine appears for your reaction-wheel actuator — and what's physically different about their actuator that puts one in theirs.
 - 2 **Why energy control alone can't balance.** You've pumped to $E = E_0$ exactly. Why does that *not* mean the rod is standing still at the top (use the level-set / contour idea)? And why does the pump command go *weak* right at the top, exactly when a balancer needs to push *hard*?
 - 3 **The three-axis catch.** The rod arrives at a perfect angle, nearly stopped, but the balancer drops it. Give the reaction-wheel-specific reason, tie it to back-EMF, and say what you'd change in the *swing-up* phase (not the balancer) to prevent it.
-

FURTHER READING

Sources

Åström & Furuta (2000), "Swinging up a pendulum by energy control," *Automatica* 36(2):287–295. Foundational — but their $\cos \theta$ is an artifact of their sideways-driven pivot, *not*

your direct-reaction wheel.

Block, Åström & Spong (2007), *The Reaction Wheel Pendulum*, Morgan & Claypool. The definitive monograph on *your exact machine* — energy swing-up via PFL, LQR balance, the switch, friction / saturation.

Spong, Corke & Lozano (2001), "Nonlinear control of the Reaction Wheel Pendulum," *Automatica* 37(11):1845–1851.

Tedrake, *Underactuated Robotics* (MIT), trajectory optimization chapter — most beginner-friendly direct collocation, for Step 6. underactuated.mit.edu/trajopt.html